

25. Java Strings | String Pool, Immutability, Internals

▼ What is a String?

Conceptually a String is a sequence of characters

▼ How does String get stored internally?

Before Java 9

Using an array of characters, but it is abstracted to the users such that they can easily perform high level operations like `concat()`, `substring()` and many more which otherwise they would always have to write custom implementations for

After Java 9

Modern java uses byte array to store the characters

▼ Which package is String a part of ?

`java.lang` package. It is autoimported and consists of other classes like `Object`, `System` etc

▼ What is an **Immutable** class in Java?

On a high level, it's a custom way of creating a class in such a way that it's internal fields can not be changed once initialized.

Once an object is constructed, it's internal state remains completely frozen throughout it's entire lifecycle

▼ What **rules** should one follow while creating an Immutable class?

1. Declate the class as final
2. Declare all fields as private and final
3. Do not provide setters
4. Onnly initialize using parametrized constructors

5. Perform defensive copying on mutable fields

▼ When should one use/create an immutable class?

→ When we need thread safety without synchronization.

Since the state can not change, multiple concurrent threads can access the object without any risk of race conditions or data corruption. Hence, no performance heavy locking mechanism is needed

→ Safe key for maps

We need consistency in hashCode creation while using HashMap and HashSets

▼ How does Java store Strings internally?

→ Since programs are heavily dependent on Strings, modern java reserves some memory known as **String Pool** for all Strings inside the Heap memory

→ Earlier this String Pool was not the part of heap memory and was located separately called as **PremGen**

▼ What is the difference between the String literal and new keyword initialisation?

→ When we create a String using the literal `" "`, it firsts checks whether the provided string already exists in the String Pool. If yes, then the reference to the existing object is returned. Else, a new String is created in the pool

→ However, when we create a String by explicitly using the `new` keyword, java creates a new object in the **Heap memory and not the String Pool!** It then afterwards, returns the reference

It is recommended to use the literals for performance gains

▼ What is the Golden rule for Strings?

→ All **compile time** strings constants with the String literal `" "` become a part of the **String Pool**

→ All the Strings created with the **new keyword** and **runtime Strings** become a part of the **Heap memory!**

▼ Why does modern java use byte array to store Strings internally?

Each char takes up 2 bytes of space. However, it was observed that under most scenarios, programmers largely used ASCII characters only which easily fits into 1 byte.

Hence, by using byte array instead, java reduced memory usage by 50%

▼ But what if programmers want to create Strings that consisted of the larger unicode set like emojis etc?

Java handles this by using a variable `int coder`

→ It can have two values, either **0** or **1**

→ If a String is declared which only contains the **ASCII** set, the value of `coder` is **0** and each character takes 1 byte of space

→ However, if there is any character which is **UNICODE**, `coder` is set to **1**

→ This means that now for every character present in the String, it would be represented by two bytes in the array

▼ In a nutshell, how has java **optimized** the String class?

1. **String Pool:** By poolings all the compile time constant strings under a specific section of the Heap Memory known as the String Pool
2. **Byte Array:** By moving from a `char[]` array to `byte[]` array, resulting in straight 50% reduction in size
3. **Caching the hash:** Calculating the hash is an expensive operation, and for something which is immutable, will always result in the same hash. Hence, we lazily initialize the hash value and store it in the `hash` variable